



Guia Prático: Automatizando com Selenium

Por Profa Lia Mariana QA

Este guia detalha o processo de criação de um script de automação de login, focando na segurança das credenciais e na robustez do código.

1. Preparação do Ambiente

Antes de começar, certifique-se de ter as ferramentas necessárias instaladas no seu terminal:

- **Verificar Python:** `python --version`.
- **Instalar Selenium:** `pip install selenium`.
- **Instalar WebDriver Manager:** `pip install webdriver-manager` (essencial para gerenciar os drivers do navegador automaticamente).
- **Instalar Python-dotenv:** `pip install python-dotenv` (necessário para ler o arquivo `.env`).

2. Configurando as Variáveis de Ambiente (.env)

Nunca escreva senhas diretamente no seu código (hardcoding). Crie um arquivo chamado `.env` na raiz do seu projeto com o seguinte conteúdo:

```
# .env
LOGIN_USER=seu_email@exemplo.com
LOGIN_PASSWORD=sua_senha_secreta
```

Adicione o arquivo `.env` ao seu `.gitignore` para garantir que ele não seja enviado para repositórios públicos.

3. O Script de Automação

Crie um arquivo chamado `automacao_login.py`. O código abaixo utiliza esperas explícitas para evitar erros de sincronização e carrega as credenciais de forma segura.

```
import os
from dotenv import load_dotenv
from selenium import webdriver
from selenium.webdriver.chrome.service import Service
from webdriver_manager.chrome import ChromeDriverManager
from selenium.webdriver.common.by import By
from selenium.webdriver.common.keys import Keys
```



```
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

# 1. Carrega as variáveis do arquivo .env
load_dotenv()
USUARIO = os.getenv("LOGIN_USER")
SENHA = os.getenv("LOGIN_PASSWORD")

# 2. Configuração do WebDriver (Download automático)
service = Service(ChromeDriverManager().install())
driver = webdriver.Chrome(service=service)

try:
    # 3. Acessar a página de login
    driver.get("https://www.exemplo.com/login") # Substitua pela URL desejada

    # 4. Espera Explícita: Aguarda o campo de e-mail aparecer
    wait = WebDriverWait(driver, 10)
    campo_email = wait.until(EC.presence_of_element_located((By.ID, 'email')))

    # 5. Preencher campos usando as variáveis de ambiente
    campo_email.send_keys(USUARIO)
    driver.find_element(By.ID, 'pass').send_keys(SENHA)

    # 6. Submeter formulário
    driver.find_element(By.ID, 'pass').send_keys(Keys.RETURN)

    print("Login executado com sucesso!")

finally:
    # 7. Fechar o navegador após a execução
    driver.quit()
```

4. Explicação dos Principais Comandos

- **By.ID, By.NAME, By.XPATH**: Métodos para localizar elementos na página (o seu "Mapa do Tesouro").
- **WebDriverWait + EC**: Garante que o Selenium espere o elemento carregar antes de interagir, evitando o erro comum de "Elemento não encontrado".
- **send_keys()**: Simula a digitação do usuário no teclado.
- **os.getenv()**: Recupera as informações sensíveis protegidas no arquivo `.env`.



5. Resolução de Problemas (Troubleshooting)

Erro	Possível Solução
Elemento não encontrado	Verifique se o ID ou XPATH do elemento mudou no site.
TimeoutException	Aumente o tempo no <code>WebDriverWait</code> ou verifique sua conexão.
Credenciais Vazias	Verifique se o arquivo <code>.env</code> está na mesma pasta do script e se os nomes das variáveis coincidem.

6. Como chamar o navegador ?

Firefox:

```
from selenium import webdriver
from webdriver_manager.firefox import GeckoDriverManager

driver =
webdriver.Firefox(executable_path=GeckoDriverManager().install())
driver.get('https://www.facebook.com')
```

Edge:

```
from selenium import webdriver
from webdriver_manager.microsoft import EdgeDriverManager

driver =
webdriver.Edge(executable_path=EdgeDriverManager().install())
driver.get('https://www.facebook.com')
```



Chrome:

```
from selenium import webdriver  
from webdriver_manager.chrome import ChromeDriverManager  
driver = webdriver.Chrome(ChromeDriverManager().install())  
driver.get('https://www.facebook.com')
```

Bônus:

```
def beneficios(driver):  
    selecionar_vinculo(driver)  
    cookie_modal(driver)  
  
    steps = [  
        ("Selecionando painel dos beneficios ...", (By.CSS_SELECTOR, '[data-testid=StarsOutlinedIcon]'), "click")  
    ]  
  
    for msg, locator, action, *args in steps:  
        print(msg)  
        elem = esperar_elemento(driver, locator)  
        if action == "click":  
            elem.click()  
        elif action == "send_keys":  
            elem.send_keys(*args)
```